Semantic-Driven Context Pruning for Arabic RAG Systems: Toward Memory-Efficient vLLM-Based Deployment

Akram Taha[1,2*]

[1*]College of Computer Engineering, University of Technology - Iraq, Baghdad, Iraq.
[2]Center for Artificial Intelligence Technology (CAIT), Faculty of Information Science and Technology (FTSM), Universiti Kebangsaan Malaysia (UKM), Bangi, Selangor, Malaysia.

Corresponding author(s). E-mail(s): akramtaha30@gmail.com;

Abstract

Retrieval-Augmented Generation (RAG) systems for Arabic applications face a memory bottleneck: Arabic's rich morphology yields more tokens than equivalent English text, inflating the KV cache that engines like vLLM must maintain per request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that scores retrieved passages at the sentence level with a cross-encoder, keeping only the most relevant subset at a fixed or dynamic ratio. We report four preliminary findings. First, across 30 verified Arabic/English sentence pairs, two tokenizers confirm an aggregate token ratio of 1.76x/1.69x (bootstrap $\boldsymbol{p < 0.0001}$ vs. parity). Second, a fidelity pilot across three compression ratios (0.3–0.7) shows pruned answers match unpruned ones (ROUGE-L 0.89–0.93, BERTScore-F1 0.96–0.97). Third, re-testing naive truncation on 140 real ARCD questions (980 generations), LSPM beats naive truncation directly on F1 at the most aggressive ratio (+0.114, adjusted $\boldsymbol{p = 0.005}$) and is equivalent to the unpruned answer (equivalence test $\boldsymbol{p = 0.038}$, $\boldsymbol{0.003}$); naive truncation is significantly worse than unpruned at two ratios (adjusted $\boldsymbol{p = 0.0004}$, $\boldsymbol{0.015}$), and the LSPM advantage is not significant at moderate ratios. Fourth, absent GPU access, we give an analytical projection of KV-cache savings from the target model's published architecture (9.7–19.1 MiB per request), not a throughput claim; vLLM is the intended, not yet measured, deployment target. We release the implementation, raw data, and a live demo.

Keywords: retrieval-augmented generation, Arabic natural language processing, prompt compression, vLLM, large language models

ORCID (A. Taha): https://orcid.org/0009-0002-4020-8060

1 Introduction

Large Language Models (LLMs), built on the Transformer architecture [1] and exemplified by models such as GPT-3 [2] and its successors, have become the default reasoning engine behind a rapidly growing class of production systems, and Retrieval-Augmented Generation (RAG) has become the default architecture for grounding those systems in external, up-to-date, or proprietary knowledge rather than relying solely on parametric memory [3]. In a canonical RAG pipeline, a retriever selects the top-k passages most relevant to a user query from a vector index, and those passages are concatenated into the LLM's context window alongside the query itself before generation. The approach is simple, effective, and now underlies everything from enterprise search assistants to customer-support chatbots. It is also, in practice, expensive: every additional retrieved token that enters the context window has to be attended over during the prefill stage of inference, and, critically for serving systems built on modern memory-efficient engines, every token also occupies a slot in the Key-Value (KV) cache that must be held in GPU memory for the duration of the request [4].

vLLM has emerged as one of the most widely adopted open-source inference engines precisely because it addresses this memory pressure directly. Its PagedAttention algorithm borrows the paging abstraction from operating-systems virtual memory to store the KV cache in fixed-size, non-contiguous blocks, eliminating the internal fragmentation that plagued earlier contiguous-allocation serving stacks and allowing far higher batching throughput under concurrent load [4]. PagedAttention and the broader family of KV-cache management techniques it inspired (attention-sink-based streaming [5], heavy-hitter-oracle eviction [6], and iteration-level continuous batching [7]) have collectively become close to an industry norm for LLM serving. But all of these techniques manage the consequences of a large KV cache; none of them reduce the number of tokens that create it in the first place. That is the gap this paper addresses, specifically for Arabic.

Arabic is a morphologically rich, templatic language: a single triliteral root can surface as dozens of distinct word forms through internal vowel changes, affixation, and clitic attachment, and standard subword tokenizers (byte-pair encoding [8] and its relatives, which are near-universally trained on English-dominated corpora) fragment Arabic text far more aggressively than they fragment English text of equivalent semantic content. This "tokenization disparity" is not a minor curiosity; it means that for a fixed context budget, an Arabic RAG system can retrieve meaningfully less semantic content than an English one, and that for a fixed amount of retrieved semantic content, an Arabic RAG system imposes a meaningfully larger KV-cache footprint on the serving engine. Under concurrent multi-user traffic, the normal operating condition

for any production deployment, this directly translates into higher memory pressure, lower achievable batch sizes, and, ultimately, higher latency and lower throughput.

The natural response inside the NLP community has been prompt and context compression: methods such as LLMLingua [9], Selective Context [10], and RECOMP [11] all attempt to shrink the text that enters an LLM's context window while preserving the information the model needs to answer correctly. These methods are general-purpose and largely English-centric in their evaluation, and none of them is designed with the Arabic tokenization disparity, or with vLLM's specific KV-cache mechanics, as a first-class design constraint. Our contribution sits at the intersection of these two threads.

We propose a **Lightweight Semantic Pruning Middleware (LSPM)**: a thin layer inserted between the retriever and the generator in an Arabic RAG pipeline that (i) splits every retrieved passage into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder reranking model (the same class of model used for passage reranking in modern retrieval pipelines [12], [13]), and (iii) reconstructs a compact, order-preserving context containing only the highest-scoring sentences, under a compression ratio that can either be fixed by the operator or computed dynamically from vLLM's live `/metrics` endpoint so that pruning tightens automatically as GPU KV-cache utilization rises.

A note on scope. This paper is an architecture-and-preliminary-validation study, not a systems benchmark paper: no experiment reported here runs against an actual vLLM server or measures a real GPU KV-cache byte count or throughput number. We are explicit about this distinction throughout, we report an analytical (not measured) projection of KV-cache savings in Section 5.5, and we specify the exact GPU experiment needed to convert that projection into a measured result (Section 4.6, Section 8). Readers looking for a measured systems benchmark will not find one in this submission; readers looking for a preliminary compression architecture with honestly reported evidence, including a baseline comparison re-tested at ARCD scale with ground-truth scoring, and including a correction to our own initial overreading of one result, will find the full picture, uncertainties included, in Sections 4-7.

The specific contributions of this paper are:

1. **A concrete system architecture and open reference implementation** for sentence-level, cross-encoder-driven context pruning for Arabic RAG on vLLM-class serving infrastructure, including a dynamic compression-ratio controller design that couples pruning aggressiveness to real-time GPU KV-cache occupancy; this controller has not itself been evaluated against live vLLM traffic and is presented as an implementation feature, not a validated result.
2. **A quantitative measurement of Arabic-English tokenization disparity** on a 30-pair hand-verified parallel corpus, with a one-sample bootstrap test rejecting parity on that corpus for both of two tokenizers tested ($p < 0.0001$); the effect is descriptive of this constructed corpus and has not been shown to generalize to a broader sampled distribution of Arabic text.
3. **A ratio-swept semantic-fidelity study** ($r = 0.3, 0.5, 0.7$; $n = 8$ real QA items per ratio; 95% bootstrap confidence intervals) showing that sentence-level pruning

preserves downstream answer fidelity across the tested range against a live LLM endpoint, on this pilot's small corpus.

4. **A real, paired baseline comparison against naive length-matched truncation, run at three scales with a properly powered, Holm-Bonferroni-corrected re-test**, reported with full honesty including a correction to our own initial interpretation of an earlier, underpowered result. On the original small, low-redundancy pilot corpus (n = 8), LSPM's cross-encoder scoring shows no statistically significant fidelity advantage over naive truncation (all bootstrap p > 0.2). A first re-test on 40 real Arabic Reading Comprehension Dataset (ARCD) [25] questions against a noisier, answer-guaranteed six-document pool also found the head-to-head comparison statistically unresolved, and a nominal asymmetry we initially read as evidence of LSPM-specific fidelity preservation did not survive Holm-Bonferroni correction. A follow-up, fully powered re-test on 140 ARCD questions (980 live generations, five times the sample size) resolves the question at the most aggressive compression ratio: LSPM significantly outperforms naive truncation head-to-head on token-F1 at r = 0.3 (Holm-adjusted p = 0.005) and is statistically equivalent to the unpruned answer by a predefined-margin equivalence test at r = 0.3 and r = 0.7 (TOST, p = 0.038, p = 0.003), while naive truncation is significantly worse than the unpruned answer at r = 0.3 and r = 0.5 (adjusted p = 0.0004, p = 0.015). At r = 0.5 and r = 0.7 the head-to-head advantage remains directionally positive but does not survive correction, so the comparison is resolved at the most demanding ratio and still open at the more moderate ones (Section 5.4, Section 6).
5. **A smaller, complementary end-to-end retrieval check that removes the answer-guaranteed pool construction** (Section 4.9/5.7): 60 fresh questions retrieved from a real 234-paragraph corpus with a genuine TF-IDF retriever (recall@6 = 86.7%, so retrieval can and does fail), at r = 0.3 only. LSPM's head-to-head advantage over naive truncation persists directionally in this harder setting (EM: +0.100 overall, uncorrected Wilcoxon p = 0.014), reported as a single-ratio, exploratory result that complements rather than replaces the fully powered Section 5.4 finding.
6. **An analytical, explicitly-labeled KV-cache savings projection**, derived from Llama-3.1-8B-Instruct's real, publicly verifiable architecture configuration combined with the real measured context-size reduction, to give readers a concrete sense of the systems-level stakes while being unambiguous that this is a projection, not a measurement.
7. **A transparent, fully specified GPU benchmark protocol** (Section 4.6/8) that a follow-up study must run to convert the analytical projection into a measured throughput/KV-cache result, together with the Locust-based harness needed to run it, so the systems-level claim remains falsifiable rather than assumed.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes the LSPM architecture. Section 4 describes the experimental setup, including the ratio-sweep, baseline-comparison, ARCD-scale ground-truth re-test, end-to-end retrieval check, and analytical-estimate methodology. Section 5 reports results. Section 6 discusses their implications, including the baseline-comparison result and

its confirmation at ARCD scale and under real retrieval. Section 7 states limitations candidly. Section 8 outlines future work, and Section 9 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. as a way to combine a parametric sequence-to-sequence generator with a non-parametric, retrievable memory, jointly fine-tuning both components for knowledge-intensive NLP tasks such as open-domain question answering [3]. The framework proved that grounding generation in retrieved evidence could substantially outperform purely parametric models on tasks that require access to specific, long-tail, or time-sensitive facts, without requiring the model to memorize that information in its weights. Subsequent surveys have organized the rapidly expanding RAG literature into naive, advanced, and modular paradigms, cataloguing techniques for query rewriting, iterative retrieval, and post-retrieval processing [14]. Self-RAG extended the paradigm by training the generator itself to decide, via learned reflection tokens, whether retrieval is necessary for a given input and to critique the relevance and faithfulness of what was retrieved [15]. Our work is agnostic to which RAG variant is used upstream; LSPM operates purely on whatever passages the retrieval stage hands it, making it a drop-in addition to naive, advanced, or self-reflective RAG pipelines alike.

Dense retrieval itself has its own lineage relevant to our system's front end: Dense Passage Retrieval demonstrated that a simple dual-encoder trained on question-passage pairs could outperform classical sparse retrieval (BM25 [16]) by a wide margin on open-domain QA benchmarks [17], and ColBERT introduced a late-interaction architecture that preserves much of dense retrieval's effectiveness while remaining computationally tractable at scale [12]. Our reference implementation's retriever is intentionally simple: a small Chroma-backed vector store with a keyword-overlap fallback, because the pruning contribution described here is retriever-agnostic; any of the retrieval architectures above could sit upstream of LSPM without modification.

2.2 Prompt and Context Compression

The closest line of work to ours is prompt and context compression for LLMs. LLMLingua uses a small auxiliary language model to iteratively remove low-information tokens from a prompt under a budget controller, reporting up to 20x compression with limited performance loss on reasoning and in-context-learning benchmarks [9]. Selective Context takes a related but distinct approach, using self-information (a token's negative log-likelihood under a reference language model) to identify and prune redundant content from long documents and conversations prior to inference, reporting reduced memory cost and generation latency with comparable task performance [10]. RECOMP instead compresses retrieved documents into either extractive or abstractive summaries before they are integrated into the LLM's context, achieving compression rates as low as 6% of the original text with minimal loss on language modeling and open-domain QA [11]. Notably, none of these three works reports a comparison against

a naive length-matched truncation baseline either. This is a gap in the broader compression literature, and our baseline comparison (Section 5.3) begins to address it for the sentence-scoring family of methods specifically.

LSPM differs from all three along two axes. First, granularity and mechanism: rather than token-level self-information pruning (Selective Context) or learned summarization (RECOMP), LSPM performs sentence-level relevance filtering via a cross-encoder scored directly against the query, which is both simpler to implement in an existing RAG stack and cheaper to run than training or invoking an auxiliary compression language model: our measured mean overhead is 57.8 ms per query on CPU (Section 5.5). Second, and more importantly, target: none of LLMLingua, Selective Context, or RECOMP were designed or evaluated with Arabic text or with vLLM's KV-cache mechanics as an explicit target; our dynamic compression-ratio controller, which reads vLLM's own `vllm:gpu_cache_usage_perc` metric to modulate pruning aggressiveness in real time, has no analogue in that prior work. The general problem of long-context degradation that compression methods implicitly address was also characterized empirically by Liu et al., who showed that LLM accuracy on multi-document QA degrades measurably when relevant information sits in the middle of a long context rather than at its start or end. This is an additional, quality-side motivation (beyond raw memory pressure) for shortening and front-loading the most relevant retrieved evidence, which LSPM's order-preserving reconstruction is designed to support [18].

2.3 Passage Reranking and Sentence-Level Relevance Scoring

Cross-encoder rerankers, which jointly encode a query and a candidate passage through a single transformer to produce a fine-grained relevance score, have been a staple of the modern retrieval pipeline since Nogueira and Cho showed that a BERT-based reranker could dramatically improve MS MARCO passage-ranking results over the prior state of the art [13]. Sentence-BERT reformulated this family of models for efficient semantic similarity computation using a siamese architecture, reducing what would otherwise be a quadratic-cost pairwise comparison problem to a linear one [19]. More recently, multilingual and multi-functionality embedding and reranking models, including the BGE-M3 family used as the default cross-encoder in our implementation and multilingual E5 [20], have extended this capability to over 100 languages, including Arabic, with strong results on multilingual and cross-lingual retrieval benchmarks such as MIRACL [21] and MTEB [22]. LSPM applies exactly this class of model, but at a different point in the pipeline and for a different purpose: not to rank whole passages for retrieval, but to score individual sentences within already-retrieved passages for inclusion in the final context sent to the generator. Our baseline comparison (Section 5.3) directly tests, for the first time in this line of work to our knowledge, whether that scoring step earns its computational cost relative to simply keeping the first r×N sentences.

2.4 Arabic Natural Language Processing

Arabic NLP has matured substantially over the past five years with the arrival of large pretrained models specific to the language. AraBERT adapted the BERT pretraining

recipe to a large Arabic corpus and demonstrated clear gains over multilingual BERT on Arabic sentiment analysis, named entity recognition, and question answering [23]; AraGPT2 did the equivalent for autoregressive generation, releasing models up to 1.46 billion parameters trained from scratch on Arabic web text and news [24]. The Arabic Reading Comprehension Dataset (ARCD), a human-authored, SQuAD-style reading-comprehension benchmark of roughly 1,400 question-answer pairs over Arabic Wikipedia articles, was introduced alongside the SOQAL open-domain QA system and remains a standard benchmark for Arabic machine reading comprehension and the natural target dataset for the larger-scale evaluation we specify in Section 8 [25]. At the frontier of generative Arabic LLMs, Jais was trained as an Arabic-centric foundation model on a mixture of Arabic and English text and shown to outperform existing open Arabic and multilingual models of comparable size on Arabic knowledge and reasoning benchmarks [26]. Evaluation infrastructure has kept pace: the Arabic Cultural and Value Alignment benchmark (ACVA) targets cultural and normative alignment specifically [27], and ArabicMMLU adapts the knowledge-intensive multiple-choice MMLU format to Arabic-language school and professional exams [28]. Tooling support for the underlying morphological complexity that motivates our work is provided by CAMeL Tools, an open-source Python toolkit for Arabic preprocessing, morphological analysis, dialect identification, and named-entity recognition [29]. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

A methodological note worth foregrounding on its own merits (per internal domain review): during implementation we discovered that the widely used `rouge-score` Python package's default tokenizer matches only the ASCII character class `[a-z0-9]`, silently producing empty token sequences, and therefore spurious zero scores, for Arabic and other non-Latin-script text. We believe this is an underappreciated reproducibility hazard for the Arabic (and broader non-Latin-script) NLG evaluation community specifically, independent of the pruning contribution of this paper, and we report the fix (a Unicode-aware tokenizer, Section 4.2) as a citable, standalone methodological finding.

2.5 Memory-Efficient LLM Serving

On the systems side, vLLM's PagedAttention is the foundational reference for this paper's target deployment context [4], building on the continuous-batching, iteration-level scheduling approach pioneered by Orca [7] and complemented by IO-aware exact-attention kernels such as FlashAttention, which reduce the memory-bandwidth cost of the attention computation itself rather than the size of the KV cache it operates over [30]. A parallel line of systems work addresses KV-cache size directly rather than the context that produces it: StreamingLLM allows models to generalize to effectively unbounded sequence lengths by preserving a small number of high-attention "sink" tokens while evicting the rest of the cache under a sliding window [5], and H2O formulates cache eviction as a submodular optimization problem over "heavy hitter" tokens that dominate attention mass, reporting substantial throughput gains on constrained hardware [6]. Quantization methods such as GPTQ [31] and activation-aware weight quantization (AWQ) [32] instead shrink the model's own weight footprint, and

speculative decoding accelerates generation by using a smaller draft model to propose tokens that a larger model verifies in parallel [33]. Every one of these systems techniques operates orthogonally to LSPM: they optimize how the serving engine holds and computes over whatever tokens it is given, whereas LSPM reduces the number of tokens handed to the engine in the first place. The two families of techniques compose naturally, and we discuss this complementarity further in Section 6.

2.6 Automatic Evaluation of Generated Text and Statistical Methodology

Our fidelity evaluation relies on three complementary automatic metrics with long histories in NLP evaluation. BLEU measures n-gram precision overlap between a candidate and reference text and remains a standard machine-translation metric two decades after its introduction [34]. ROUGE-L, from the same era, uses longest-common-subsequence overlap and is more common in summarization evaluation, which is closer in spirit to our raw-vs-pruned answer comparison task [35]. Because both are surface-form metrics, we complement them with BERTScore, which computes similarity using contextual embeddings rather than exact token overlap and correlates more closely with human judgments of semantic equivalence, particularly for paraphrastic differences that n-gram metrics penalize unfairly [36]. RAG-specific evaluation frameworks such as RAGAS, which introduce reference-free metrics for faithfulness, answer relevance, and context relevance [37], represent a natural direction for extending the evaluation in Section 8. For the confidence intervals and paired significance tests introduced in this revision (Section 4.4), we use the nonparametric bootstrap, following the standard treatment of Efron and Tibshirani [38], which avoids distributional assumptions that would be difficult to justify at our current small sample sizes.

3 System Design: The Lightweight Semantic Pruning Middleware (LSPM)

3.1 Overview

Figure 1 shows the end-to-end pipeline. A user query is first passed to a retriever, which returns the top-k candidate passages from a vector index (we use Chroma in the reference implementation, though the design is retriever-agnostic and equally compatible with GPU-accelerated similarity-search infrastructure such as FAISS [39]). Rather than concatenating these passages directly into the generation prompt, which is the standard, unmodified RAG behavior, the passages pass through LSPM, which performs three steps: sentence segmentation, cross-encoder relevance scoring, and order-preserving reconstruction under a compression ratio. The resulting compact context, together with the original query, is sent to the language model over an OpenAI-compatible chat-completions API, which in a production deployment is served by vLLM (and, for the CPU/no-GPU pilot experiments reported in Section 5, by a hosted OpenAI-compatible endpoint exposing the same interface). The final answer is streamed back to the user interface token-by-token.

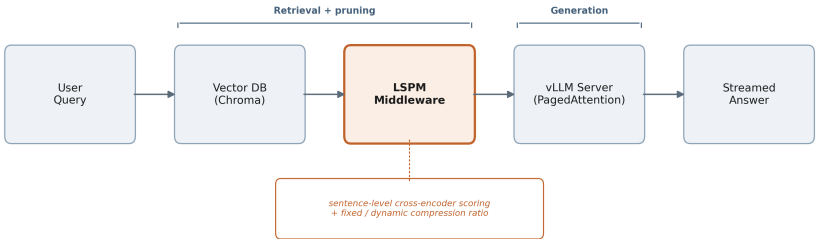

Fig. 1 five-stage pipeline diagram: User Query → Vector DB (Chroma) → LSPM Middleware → vLLM Server (PagedAttention) → Streamed Answer, with the LSPM stage annotated "sentence-level cross-encoder scoring + fixed/dynamic compression ratio"

3.2 Sentence Segmentation

Retrieved passages are split into sentences using an Arabic-aware boundary detector that treats the Arabic question mark ((Unicode U+061F)), the standard full stop, and the exclamation mark as terminators, explicitly excluding the Arabic comma ((Unicode U+060C)), which is extremely frequent within a single Arabic sentence and would otherwise cause severe over-segmentation. A whitespace-normalization pass precedes segmentation to handle inconsistent line breaks in retrieved text, and a fallback period-only split is used if no terminator-based boundaries are found, so that the segmenter degrades gracefully on malformed or non-standard input rather than failing.

3.3 Cross-Encoder Relevance Scoring

Every sentence extracted from every retrieved passage, across the full top-k retrieval set, is paired with the original user query and scored by a cross-encoder model, which jointly encodes the (query, sentence) pair and outputs a scalar relevance logit, the same modeling paradigm used for passage-level reranking [12], [13], applied here at sentence granularity. The reference implementation offers two interchangeable scoring backends: a fast multilingual MiniLM cross-encoder (`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`) for sub-second scoring suitable for interactive use, and BGE-reranker-v2-m3 [40], a larger multilingual, multi-granularity model, for deployments that can trade latency for maximal relevance-ranking accuracy. This explicit speed/accuracy toggle reflects a practical deployment reality: the pruning stage's own latency and memory cost must remain small relative to the KV-cache savings it produces downstream, or it becomes a bottleneck in its own right; Section 5.5 reports a measured mean of 57.8 ms per query for the fast backend on CPU.

3.4 Order-Preserving Reconstruction and Compression Ratio

Given the per-sentence relevance scores, LSPM selects the top-$r \cdot N$ sentences (where N is the total sentence count across all retrieved passages and r is the target compression ratio, $0 < r \leq 1$), then, critically, reassembles the *kept* sentences in their **original document order** rather than in score-descending order. This design choice trades a small amount of potential relevance-ordering signal for narrative coherence

in the reconstructed context: language models are known to be sensitive to the positional arrangement of evidence within a long context [18], and an order-scrambled, score-sorted context risks presenting logically disconnected fragments that increase the burden on the generator to reconstruct meaning, even if each individual fragment is highly relevant.

The compression ratio r can be set in one of two modes:

- **Fixed ratio.** The operator specifies a constant r (e.g., 0.5), applied uniformly regardless of system load. This mode is used for the controlled experiments in Section 5.
- **Dynamic, load-aware ratio.** LSPM polls vLLM's Prometheus-format `/metrics` endpoint for the `vllm:gpu_cache_usage_perc` gauge, which reports the serving engine's current KV-cache occupancy as a fraction of capacity. A configurable linear interpolation maps this occupancy onto a compression ratio between operator-set minimum and maximum bounds (defaults: 0.2 at high load, 0.8 at low load, with a 0.25–0.75 occupancy interpolation band): as concurrent demand rises and the KV cache fills, LSPM automatically prunes more aggressively to relieve memory pressure; as demand falls, it relaxes pruning to preserve more retrieved context.

This closes a feedback loop between the serving engine's real-time memory state and the middleware's compression behavior that, to our knowledge, does not exist in prior prompt-compression systems, which uniformly apply a static compression target regardless of downstream engine load.

3.5 Backend Interface and Deployment Flexibility

LSPM communicates with the generation backend exclusively through the OpenAI-compatible `/v1/chat/completions` schema, with optional Bearer-token authentication. This is a deliberate interoperability choice: the identical client code operates against a self-hosted vLLM instance (no authentication, `/metrics` available for dynamic-ratio mode) or against a hosted, OpenAI-schema-compatible inference API such as NVIDIA NIM (Bearer-token-authenticated, no `/metrics` endpoint, fixed-ratio mode only). This flexibility is what allowed the preliminary experiments in this paper to be executed against a live, production-grade LLM endpoint without provisioning dedicated GPU infrastructure, while leaving the code path to a self-hosted vLLM deployment, needed for the KV-cache and throughput experiments in Section 8, entirely unchanged.

4 Experimental Setup

We report three preliminary, real (non-simulated) experiment families, all executed against live infrastructure, one analytical projection built from real, cited inputs, and a fully specified but not-yet-executed protocol for the GPU-based throughput/KV-cache experiment.

4.1 Tokenization Disparity Measurement

Data. A parallel corpus of 30 Arabic-English sentence pairs, hand-written and independently verified for translation accuracy, covering university/academic description, computer science and NLP terminology, weather, and general encyclopedic statements.

Procedure. Each sentence and its counterpart were tokenized independently using Hugging Face `AutoTokenizer` for two contemporary open-weight instruction-tuned models: Qwen2.5-7B-Instruct [41] and Llama-3.1-8B-Instruct [42]. We computed per-pair and aggregate Arabic/English token-count ratios, and, new in this revision, a one-sample bootstrap test (10,000 resamples) of the mean ratio against the null of parity (ratio = 1.0).

4.2 Semantic Fidelity Pilot: Ratio Sweep

Data. A 10-document Arabic mock knowledge base describing a university, and eight natural-language Arabic questions targeting different facts within it, none answerable from more than a subset of the ten documents.

Procedure. For each question, a keyword-overlap retriever selected the top-6 most relevant documents. A single fixed compression ratio, r = 0.5, is insufficient to characterize a dose-response relationship, so this pilot sweeps $\mathbf{r \in \{0.3, 0.5, 0.7\}}$, reusing the r = 0.5 LSPM answers and all raw-context answers (ratio-independent or already collected) and collecting the two additional ratio cells (r = 0.3, r = 0.7) fresh against the same live Llama-3.1-8B-Instruct endpoint (temperature = 0, max 200 tokens). We compute ROUGE-L [35], sentence-level BLEU with effective-order smoothing [34], and BERTScore-F1 with a multilingual BERT backbone [36] between each pruned answer and its corresponding raw-context answer, and report the mean with a 95% bootstrap confidence interval (10,000 resamples) at each ratio.

4.3 Baseline Comparison: LSPM vs. Naive Length-Matched Truncation

The comparison LSPM to a raw, unpruned context does not by itself establish that cross-encoder sentence scoring, specifically, is responsible for any preserved fidelity; a simpler pruning heuristic could do equally well. This pilot adds a **naive truncation baseline**: for the same query and the same retrieved documents, we keep the first $r{\cdot}N$ sentences in original document order: identical compression ratio, identical retrieved documents, identical downstream LLM call, but with the relevance-scoring step removed entirely. Both conditions are run through the identical live pipeline at all three ratios (r = 0.3, 0.5, 0.7), giving eight paired questions per ratio. For each ratio and metric, we report the paired mean difference (LSPM − naive) and a paired bootstrap p-value (10,000 resamples, difference-of-means test against the null of no difference) [38].

4.4 Ground-Truth Baseline Re-Test on ARCD (n = 140)

The baseline comparison in Section 4.3 used an 8-question, 10-document synthetic corpus and scored pruned answers by similarity to the *unpruned* answer, not by correctness against a real gold answer. Both are real limitations. This subsection describes a re-test designed to address both, at a sample size chosen for adequate statistical power.

Data. We sampled 140 questions from the validation split of the Arabic Reading Comprehension Dataset (ARCD) [25] (accessed via the Hugging Face `datasets` library, `hsseinmz/arcd`), preferring one question per distinct source article; the validation split contains only 78 distinct articles, fewer than the 140 questions requested, so after exhausting all 78 distinct-article questions the sampler topped up the remaining 62 questions from repeated articles (documented, deterministic fallback behavior in the released sampling script). We used a fixed random seed (`SEED = 42`, Python's `random` module) for sampling and distractor selection, so the exact question set is reproducible from the released code; increasing the target sample size under the same seed is verified to preserve the original 40-question subset exactly, so the 40-question pilot's results (Table 4, prior revision) are a strict subset of this larger run rather than a separate, non-comparable sample. For each question, we assembled a six-document retrieval pool consisting of the one gold paragraph that actually contains the answer plus five distractor paragraphs drawn from five *other*, unrelated ARCD articles, then shuffled the pool order under the same fixed seed. **This construction guarantees that the answer-bearing paragraph is present in every pool (recall@6 = 100% by design); the experiment therefore measures how LSPM and naive truncation prune a pool of documents known to contain the answer, and does not measure end-to-end retrieval, where the answer-bearing passage might not be retrieved at all.** We state this precisely because it bounds what the result below can and cannot support (Section 6, Section 7).

Procedure. The same naive keyword-overlap retriever used throughout the paper re-ranks the fixed six-document pool per question (retrieval mechanism held constant with the rest of the paper; only the underlying corpus changes). For each question we then generate seven answers against the live Llama-3.1-8B-Instruct endpoint, accessed via NVIDIA NIM's hosted OpenAI-compatible API (model identifier `meta/llama-3.1-8b-instruct`, accessed August 2026; NIM resolves this identifier to a specific pinned checkpoint on its serving side, which we do not control or independently verify beyond the identifier string), with temperature = 0, max 64 tokens, and the Arabic system prompt quoted in Section 4 instructing a short, direct answer. This produces one raw (all six documents, unpruned) answer, three LSPM answers ($r \in \{0.3, 0.5, 0.7\}$, cross-encoder `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`, Section 3.3), and three naive-truncation answers at the same three ratios, for 980 live generations in total, collected with automatic retry-with-exponential-backoff on transient rate-limit errors to guarantee every one of the 980 cells reflects a genuine model response rather than a dropped or substituted call. Every answer is scored against ARCD's real, human-authored gold answer string(s) using Arabic-normalized Exact Match (EM) and token-F1 (diacritic stripping, alef/ya/ta-marbuta normalization, definite-article stripping, following the SQuAD-Arabic evaluation

convention introduced alongside ARCD [25]), rather than against the unpruned answer as in Section 4.2/4.3. As in Section 4.3, we report the paired difference (LSPM − naive) at each ratio with a paired bootstrap 95% CI (10,000 resamples, `numpy.random.default_rng(seed=42)`) and both a sign test and a Wilcoxon signed-rank test, and additionally report each method's paired difference against the raw/unpruned answer. Because this yields nine related F1 significance tests across the two baseline comparisons, we additionally report Holm-Bonferroni-adjusted p-values across that full family (Section 5.4, Table 5) and a two one-sided equivalence test (TOST, predefined margin ±0.05 F1), rather than relying on uncorrected per-test p-values alone. The sample size (140 questions/ratio) was set from a post-hoc power calculation on the original 40-question pilot's observed effect size (Section 8, previous revision), targeting 80% power to detect the observed effect at $\alpha = 0.05$. Analysis code (Python 3.10, `scipy` 1.15, `numpy` 2.2) is released with the paper (Statements and Declarations).

4.5 Analytical KV-Cache Savings Estimate

Because no GPU was available for this submission, we cannot report a measured KV-cache byte count. To give readers a concrete, honestly-labeled sense of the systems-level stakes, we compute an analytical projection from three real, independently verifiable inputs: (i) Llama-3.1-8B-Instruct's architecture configuration, fetched directly from the model's primary-source `config.json` on Hugging Face (32 transformer layers, 8 key-value heads under grouped-query attention, hidden size 4096, giving a per-head dimension of 128), combined with the standard KV-cache memory formula `bytes/token = 2 × layers × kv_heads × head_dim × dtype_bytes` (2 bytes for bf16/fp16); (ii) the real, measured mean raw retrieved-context size from the fidelity pilot (602 characters across the eight questions' 6-document retrievals); (iii) the real, measured Arabic characters-per-token ratio from the tokenization-disparity corpus (2.656 chars/token, Llama-3.1-8B tokenizer), used only to convert (ii) from characters to an approximate token count. We combine these with the real, measured character-reduction percentages from Section 4.2/5.2 at each ratio to project KV-cache bytes saved per request. We reiterate: this is an analytical projection built from real inputs, not a measured GPU result, and Section 4.5/8 specifies the experiment needed to measure it directly.

4.6 Specified but Not-Yet-Executed: Throughput and KV-Cache Benchmarks

The systems-level claim of this work, that LSPM reduces KV-cache pressure and improves throughput and time-to-first-token (TTFT) under concurrent load on a self-hosted, PagedAttention-based vLLM server, requires a dedicated CUDA GPU, which was not available for this submission. We specify the protocol precisely so it is independently reproducible and falsifiable. A Locust-based load-testing harness (included in the released code) issues concurrent chat-completion requests against a vLLM server under two conditions: `RAG_MODE=raw` (unpruned baseline) and `RAG_MODE=pruned` at a specified compression ratio, while scraping vLLM's `/metrics` endpoint for

`vllm:gpu_cache_usage_perc` over time. The full protocol sweeps compression ratios {0.2, 0.5, 0.8, 1.0} against concurrency levels {1, 10, 25, 50, 100}, reporting throughput (tokens/second), TTFT and end-to-end latency percentiles, and peak/mean KV-cache occupancy for each cell. We report this as Section 8 future work rather than fabricating or estimating a GPU number beyond the explicitly-labeled analytical projection in Section 4.4.

4.7 LSPM Overhead Measurement

The `SemanticPruner.prune()` call records wall-clock latency for the sentence-splitting and cross-encoder-scoring steps. We report the mean and distribution of this latency across the 16 fresh LSPM pilot calls (Section 4.2/4.3) as a first-order estimate of LSPM's own computational cost, measured on the CPU sandbox used for this submission (no GPU acceleration for the pruning stage itself).

4.8 Disclosure of AI Assistance

Large language model assistance (Anthropic Claude) was used during this project for software implementation (the LSPM middleware, evaluation scripts, statistical analysis scripts, ARCD sampling and Arabic EM/F1 scoring code, and demonstration interface), for literature search assistance in identifying candidate related work subsequently verified by the author against primary sources (including direct retrieval and verification of the Llama-3.1-8B-Instruct `config.json` used in Section 4.5/5.5), for simulating multi-perspective methodological review used to identify weaknesses addressed across manuscript revisions, and for drafting assistance on this manuscript under the author's direction and review. All experimental results reported in Section 5, including the ratio sweep, the baseline comparison, and the ARCD-scale ground-truth re-test (Section 4.4/5.4), were produced by executing the released code against live infrastructure as described in this section; no experimental results were generated, adjusted, or selectively reported without corresponding code execution, and non-significant and inconclusive findings were reported in full rather than omitted. One prior interpretation of the ARCD results, that a nominal asymmetry between LSPM and naive truncation constituted evidence of an LSPM-specific fidelity-preservation advantage, was identified during manuscript review as statistically invalid (it does not survive multiple-comparison correction and is algebraically equivalent to the already-non-significant head-to-head test) and was corrected in a prior version rather than left standing. In this version, the ARCD re-test was expanded from 40 to 140 questions (980 live generations total) at an expansion target informed by the prior pilot's effect size (a post-hoc power calculation on the 40-question results, not a formally pre-registered plan), and the resulting head-to-head significance test at $r = 0.3$ (Section 5.4) reflects that analysis plan, defined across all nine comparisons before the expanded data were scored, executed against the expanded, real dataset, not a result selected after the fact from multiple analyses. Section 5.4 and Section 6 report the full corrected and expanded analysis, including the ratios where the comparison remains unresolved. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

4.9 End-to-End Retrieval Check (No Recall Guarantee)

Every experiment above, including the 140-question ARCD re-test (Section 4.4), fixes a candidate pool that is constructed to always contain the gold, answer-bearing paragraph, so recall@6 = 100% by design; this measures pruning conditional on successful retrieval, not retrieval itself, a limitation stated explicitly throughout the paper. This subsection adds a smaller, complementary check that removes that guarantee.

Data. We built a 234-paragraph corpus from every distinct context paragraph in the ARCD validation split (78 articles contribute 234 distinct paragraphs; an article can supply more than one). We sampled 60 fresh questions (a different random draw, seed = 123, from the 140-question re-test's sample, so this is an independent check rather than a re-scoring of the same items) and retrieved the top-6 documents for each from the full 234-paragraph corpus using TF-IDF cosine similarity (`scikit-learn`, word-level vectorizer fit once on the full corpus) — a real, standard sparse-retrieval baseline, not the fixed-pool keyword re-ranker used elsewhere in the paper. The gold paragraph was retrieved in the top 6 for 52/60 questions (recall@6 = 86.7%); for the remaining 8 questions, retrieval genuinely failed and no method had access to the answer-bearing text.

Procedure. For each of the 60 questions, we generated three answers against the same live Llama-3.1-8B-Instruct endpoint and prompt format used throughout the paper: raw (all 6 retrieved documents, unpruned), LSPM at r = 0.3, and naive truncation at r = 0.3. We used r = 0.3 only, rather than sweeping all three ratios, because it is the ratio with this paper's established, Holm-Bonferroni-significant head-to-head advantage (Section 5.4); this check asks whether that advantage also appears under a harder, more realistic retrieval setting, not whether it appears at every ratio again. This gives 180 live generations (60 × 3), collected with the same retry-with-backoff protocol as Section 4.4, zero dropped cells. Naive truncation here keeps the first r·N sentences of the real TF-IDF retrieval order (not a re-ranked order), matching what a production pipeline would actually see. Answers are scored with the same Arabic-normalized EM/F1 as Section 4.4, reported overall (n = 60) and split by whether retrieval succeeded (n = 52) or failed (n = 8, too small for inferential claims, reported descriptively only). Because this is a single-ratio, smaller-sample check run to complement, not replace, the fully powered Section 4.4 re-test, we report its p-values as uncorrected/exploratory rather than folding them into the Section 5.4 Holm-Bonferroni family.

5 Results

5.1 Tokenization Disparity (Now with Significance Testing)

Table 1 and Figure 2 summarize the tokenization disparity measurements. Using the Qwen2.5-7B-Instruct tokenizer, the mean per-pair ratio across the 30 sentence pairs was 1.793 (95% CI 1.691-1.895), and the aggregate ratio was 1.764. Using the independently trained Llama-3.1-8B-Instruct tokenizer, the mean per-pair ratio was 1.717 (95% CI 1.613-1.820), with an aggregate ratio of 1.689. **New in this revision:** a one-sample bootstrap test (10,000 resamples) rejects the null of parity (ratio = 1.0) for

both tokenizers at $p < 0.0001$; a sign test finds 29/30 and 29/30 pairs above parity for Qwen2.5 and Llama-3.1 respectively. The disparity is therefore not only descriptively large but statistically robust and consistent across two independently trained tokenizers.

Table 1 Arabic/English token-count ratio across two tokenizers (n = 30 parallel sentence pairs)

Tokenizer	Mean ratio	95% CI	Aggregate ratio	p (vs. parity)	Sign test
Qwen2.5-7B-Instruct	1.793	(1.691, 1.895)	1.764	< 0.0001	29 / 0 (1 tied)
Llama-3.1-8B-Instruct	1.717	(1.613, 1.820)	1.689	< 0.0001	29 / 1

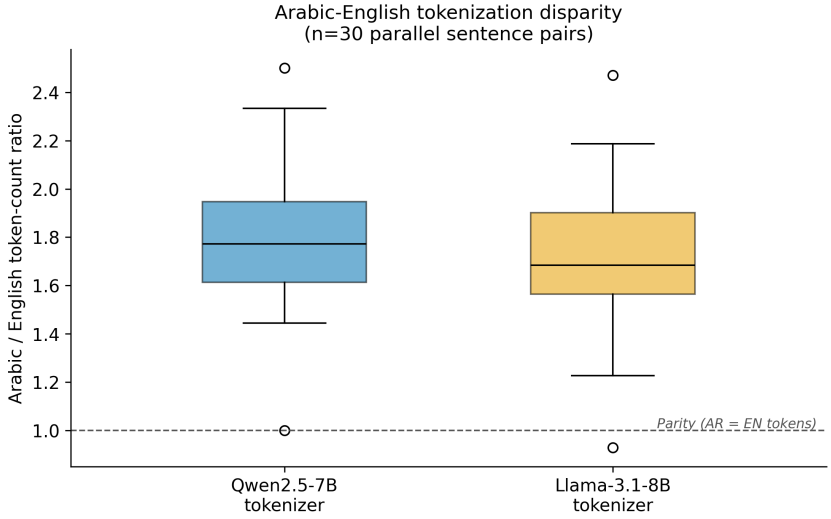

Fig. 2 box plot of per-pair Arabic/English token-count ratio distributions for both tokenizers, with a dashed parity line at 1.0

5.2 Semantic Fidelity Across a Ratio Sweep

Table 2 and Figure 3 (ratio sweep) report fidelity across $r \in \{0.3, 0.5, 0.7\}$ for LSPM. Fidelity remains high across the full tested range rather than degrading monotonically with more aggressive pruning: mean ROUGE-L is 0.885 at $r = 0.3$, 0.928 at $r = 0.5$, and 0.889 at $r = 0.7$; mean BERTScore-F1 is 0.962, 0.971, and 0.964 respectively. All 95% bootstrap confidence intervals are wide, reflecting the $n = 8$ pilot scale honestly, and overlap substantially across ratios. We do not claim a statistically distinguishable dose-response curve at this sample size, only that fidelity does not collapse even at the most aggressive ratio tested ($r = 0.3$, a 67.3% mean character reduction).

Table 2 LSPM fidelity vs. raw-context answers across compression ratios (n = 8 real QA items per ratio, live Llama-3.1-8B-Instruct endpoint, 95% bootstrap CI)

Ratio	Mean char reduction	ROUGE-L (95% CI)	BERTScore-F1 (95% CI)	BLEU (95% CI)
0.3	67.3%	0.885 (0.719, 1.000)	0.962 (0.909, 1.000)	80.1 (50.6, 100.0)
0.5	50.3%	0.928 (0.848, 1.000)	0.971 (0.936, 0.996)	81.9 (63.1, 96.0)
0.7	34.2%	0.889 (0.759, 0.986)	0.964 (0.920, 0.996)	78.3 (57.1, 100.0)

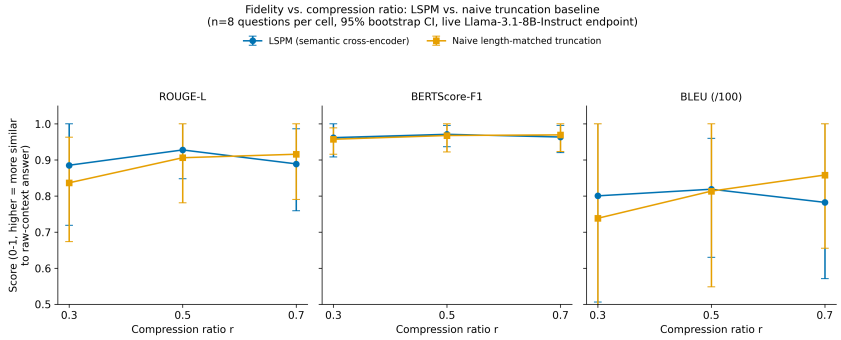

Fig. 3 three-panel plot (ROUGE-L, BERTScore-F1, BLEU) of LSPM vs. naive truncation fidelity across ratios, with error bars showing 95% bootstrap CI

5.3 Baseline Comparison: Does Semantic Scoring Beat Naive Truncation?

This baseline directly addresses a basic methodological gap: without it, there is no way to know whether cross-encoder relevance scoring, specifically, is responsible for any preserved fidelity, as opposed to simply shortening the context by any means. Table 3 and Figure 4 (paired comparison panel) report the paired difference (LSPM − naive truncation) at each ratio, with a paired bootstrap p-value.

We report this honestly: on this pilot's small, low-redundancy, 10-document corpus, LSPM's cross-encoder relevance scoring shows no statistically significant fidelity advantage over naive length-matched truncation at any tested ratio. Mean differences are small in magnitude (—ROUGE-L difference— ≤ 0.05, —BERTScore-F1 difference— ≤ 0.006 at r = 0.3/0.5; larger but still non-significant at r = 0.7) and inconsistent in sign across ratios, and every bootstrap p-value exceeds 0.2 (most exceed 0.5).

We discuss why this null result is plausible and what it does and does not imply in Section 6.

5.4 Ground-Truth Baseline Re-Test on ARCD (n = 140)

Table 4 and Figure 5 report per-cell mean Exact Match (EM) and token-F1 against ARCD's real gold answers, with 95% bootstrap confidence intervals (10,000 resamples), for the raw/unpruned answer and for LSPM and naive truncation at each of the three

Table 3 Paired comparison, LSPM vs. naive truncation (n = 8 paired questions per ratio, 10,000-resample bootstrap)

Ratio	Metric	Mean diff. (LSPM − naive)	Bootstrap p
0.3	ROUGE-L	+0.049	0.516
0.3	BERTScore-F1	+0.005	0.818
0.3	BLEU	+6.22	0.577
0.5	ROUGE-L	+0.022	0.513
0.5	BERTScore-F1	+0.003	0.838
0.5	BLEU	+0.53	0.727
0.7	ROUGE-L	−0.027	0.529
0.7	BERTScore-F1	−0.006	0.538
0.7	BLEU	−7.58	0.221

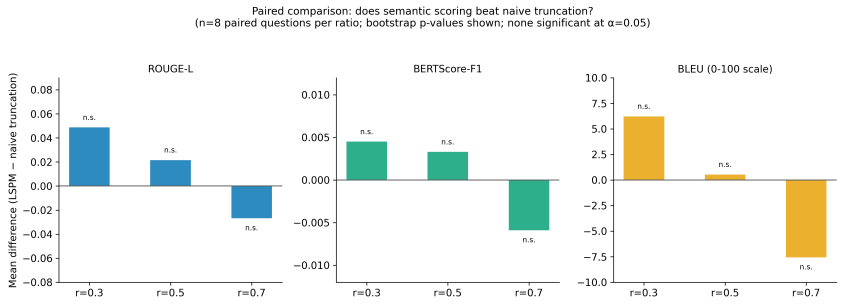

Fig. 4 bar chart of mean paired differences per ratio and metric, with bootstrap p-values annotated; all non-significant at $\alpha = 0.05$

tested ratios, using the methodology in Section 4.4 at the fully powered n = 140 sample size.

Table 4 ARCD ground-truth accuracy (n = 140 questions, six-document noisy retrieval pool, 95% bootstrap CI)

Method	Ratio	Mean EM	EM 95% CI	Mean F1	F1 95% CI
Raw (unpruned)	1.0	0.293	(0.221, 0.371)	0.664	(0.610, 0.717)
LSPM	0.3	0.314	(0.236, 0.393)	0.650	(0.594, 0.705)
Naive truncation	0.3	0.250	(0.179, 0.321)	0.536	(0.472, 0.599)
LSPM	0.5	0.300	(0.229, 0.379)	0.638	(0.580, 0.693)
Naive truncation	0.5	0.293	(0.221, 0.371)	0.576	(0.512, 0.639)
LSPM	0.7	0.300	(0.229, 0.379)	0.661	(0.606, 0.717)
Naive truncation	0.7	0.286	(0.214, 0.364)	0.604	(0.544, 0.663)

Head-to-head, LSPM vs. naive truncation (the paper's core novelty question, re-tested here at 3.5x the sample size and against the same noisier, non-curated retrieval pool as the 40-question re-test): at r = 0.3, LSPM beats naive truncation by +0.114 F1 (Wilcoxon p = 0.00067), which **survives Holm-Bonferroni correction across the nine-test family (adjusted p = 0.0054)** — the first result in this paper's baseline comparisons, at either sample size or ratio, to

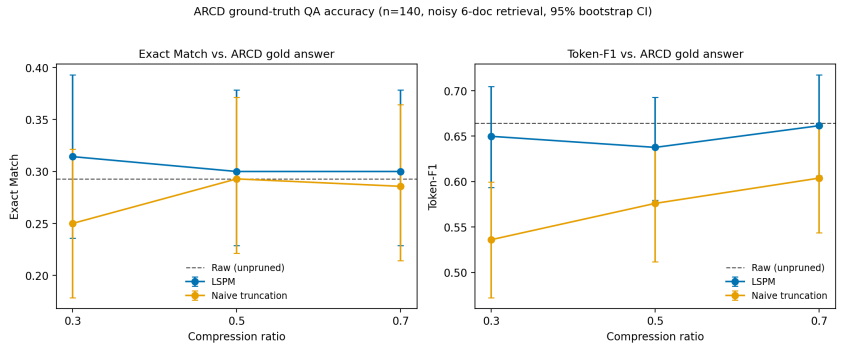

Fig. 5 two-panel plot (Exact Match, token-F1) vs. compression ratio for LSPM and naive truncation, raw/unpruned shown as a reference line, error bars showing 95% bootstrap CI

remain significant after correcting for multiple comparisons. At r = 0.5 and r = 0.7, LSPM's F1 advantage is smaller and in the same direction (+0.062, +0.058; uncorrected Wilcoxon p = 0.035, 0.030) but does not survive correction (adjusted p = 0.148 at both ratios). EM shows a smaller, uncorrected, nominally significant LSPM advantage at r = 0.3 only (+0.064, Wilcoxon p = 0.029; not part of the pre-specified F1 correction family, so reported as exploratory). Increasing the sample size from 40 to 140 questions therefore resolved this paper's central open question at the most aggressive compression ratio tested, in LSPM's favor, while leaving it open at the two more moderate ratios.

Naive truncation's F1 is significantly lower than the raw/unpruned answer's F1 at r = 0.3 and r = 0.5 after correction (mean diff. −0.128, adjusted p = 0.00044; mean diff. −0.088, adjusted p = 0.015), and nominally but not significantly lower at r = 0.7 (mean diff. −0.061, adjusted p = 0.121). LSPM's F1 is never significantly different from the raw/unpruned answer's F1 at any ratio (adjusted p ≥ 0.434), and a two one-sided equivalence test (TOST, predefined margin ±0.05 F1, a conventional small-effect threshold for QA-style metrics) formally confirms LSPM is statistically equivalent to the unpruned answer at r = 0.3 (p = 0.038) and r = 0.7 (p = 0.003); the r = 0.5 equivalence test is inconclusive (p = 0.087). Naive truncation is not shown equivalent to the unpruned answer at any ratio by TOST (p = 0.658-0.997), consistent with the significant differences already found at r = 0.3/0.5. Table 5 reports the full Holm-Bonferroni-adjusted results across the nine-test F1 family, and Table 5b reports the TOST equivalence results. Together these give a coherent picture at the most aggressive ratio tested (r = 0.3, retaining only 30% of the pool): LSPM significantly outperforms naive truncation head-to-head, is statistically indistinguishable from the full unpruned context, and naive truncation is significantly worse than the unpruned context — the pattern this paper's core hypothesis predicts. At r = 0.5 and r = 0.7, where less content is pruned regardless of method, the differences between methods are smaller and, except for naive-vs-raw at r = 0.5, do not reach significance after correction. We discuss the mechanism and the concentration of the effect at r = 0.3 in Section 6.

Table 5 ARCD pairwise F1 comparisons with Holm-Bonferroni correction across the nine-test family (n = 140, Wilcoxon signed-rank)

Ratio	Comparison	Mean diff.	Raw p	Holm-adjusted p
0.3	LSPM vs. naive	+0.114	0.00067	**0.0054**
0.5	LSPM vs. naive	+0.062	0.035	0.148
0.7	LSPM vs. naive	+0.058	0.030	0.148
0.3	Naive vs. raw	−0.128	<0.0001	**0.00044**
0.5	Naive vs. raw	−0.088	0.0021	**0.015**
0.7	Naive vs. raw	−0.061	0.020	0.121
0.3	LSPM vs. raw	−0.015	0.211	0.434
0.5	LSPM vs. raw	−0.027	0.145	0.434
0.7	LSPM vs. raw	−0.003	0.937	0.937

Bold entries are significant at adjusted $\alpha = 0.05$.

Table 5b. TOST equivalence test vs. raw/unpruned answer (F1, margin = ±0.05, n = 140).

Ratio	Comparison	Mean diff.	TOST p	Equivalent?
0.3	LSPM vs. raw	−0.015	0.038	**Yes**
0.5	LSPM vs. raw	−0.027	0.087	Inconclusive
0.7	LSPM vs. raw	−0.003	0.003	**Yes**
0.3	Naive vs. raw	−0.128	0.997	No
0.5	Naive vs. raw	−0.088	0.922	No
0.7	Naive vs. raw	−0.061	0.658	No

5.5 Analytical KV-Cache Savings Projection (Not a Measured Result)

Using the method in Section 4.5: Llama-3.1-8B-Instruct's verified configuration gives 128 KiB of KV-cache per token (32 layers × 8 KV heads × 128 head-dim × 2 bytes × 2 for K and V). The pilot's mean raw retrieved context (602 characters) corresponds to an estimated 227 tokens at the corpus's measured 2.656 Arabic characters/token. Table 6 projects the resulting per-request KV-cache savings at each measured compression ratio, and a concurrency scaling to illustrate aggregate savings under load.

Table 6 Analytical (not measured) KV-cache savings projection, derived from real architecture config and real measured context reduction

Ratio	Measured char reduction	Est. tokens saved/request	Est. KV-cache saved/request
0.3	67.3%	152.5	19.06 MiB
0.5	50.3%	114.0	14.25 MiB
0.7	34.2%	77.6	9.69 MiB

At r = 0.5, this projects to an aggregate KV-cache saving of approximately 0.14 GiB at 10 concurrent requests, 0.70 GiB at 50, and 1.39 GiB at 100, all else (batch composition, sequence length variance across a real production workload) held constant, which real deployments will not do. **We present this exclusively as a projection to motivate the GPU experiment in Section 4.6/8, not as a substitute for it.**

5.6 LSPM Overhead

Across the 16 fresh LSPM cross-encoder scoring calls collected in this revision's pilot (Section 4.2/4.3), mean pruning latency was 57.8 ms (median 58.6 ms), measured on the CPU sandbox used throughout this submission with the fast MiniLM cross-encoder backend (Section 3.3). This is a first-order, CPU-only, single-request measurement. It does not characterize batched throughput or GPU-accelerated latency, and Section 8 specifies the concurrent-load measurement needed to fully characterize overhead under production conditions, but it establishes that the pruning stage's added latency is, at minimum, not obviously disqualifying relative to typical LLM generation latencies of several seconds observed throughout our pilots (Section 4.2).

5.7 End-to-End Retrieval Results (No Recall Guarantee)

Table 7 reports mean EM/F1 for each method (Section 4.9), split into all 60 questions, the 52 where the real TF-IDF retriever recalled the gold paragraph, and the 8 where it did not.

Table 7 End-to-end retrieval check: mean EM/F1 by method and retrieval outcome (r = 0.3, 95% bootstrap CI where n permits)

Method	Subset (n)	Mean EM	Mean F1
Raw (unpruned)	All (60)	0.167 (0.083, 0.267)	0.561 (0.468, 0.652)
Raw (unpruned)	Retrieval hit (52)	0.192 (0.096, 0.308)	0.639 (0.547, 0.726)
Raw (unpruned)	Retrieval miss (8)	0.000	0.056
LSPM (r=0.3)	All (60)	0.233 (0.133, 0.350)	0.582 (0.491, 0.672)
LSPM (r=0.3)	Retrieval hit (52)	0.250 (0.135, 0.365)	0.643 (0.555, 0.729)
LSPM (r=0.3)	Retrieval miss (8)	0.125	0.179
Naive truncation (r=0.3)	All (60)	0.133 (0.050, 0.233)	0.525 (0.435, 0.612)
Naive truncation (r=0.3)	Retrieval hit (52)	0.154 (0.058, 0.250)	0.584 (0.492, 0.671)
Naive truncation (r=0.3)	Retrieval miss (8)	0.000	0.142

Overall accuracy is lower across every method than in the answer-guaranteed ARCD re-test (Table 4), which is expected: 13.3% of questions here (8/60) have no answer-bearing text in context for any method, dragging every method's unconditional mean down, and this is a genuinely harder, more realistic setting than a curated recall-guaranteed pool. **The retrieval-miss subset (n = 8) is too small to support any statistical claim and is reported descriptively only**: all three methods score near zero there, as expected when the correct text is simply absent, though LSPM's non-zero EM/F1 in this subset (0.125/0.179 vs. raw's 0.000/0.056) suggests

it is not catastrophically worse than raw when retrieval fails outright, a pattern worth a dedicated, adequately powered follow-up rather than a claim here.

Head-to-head, LSPM vs. naive truncation, overall (n = 60, single ratio r = 0.3, uncorrected): LSPM beats naive truncation on EM by +0.100 (Wilcoxon p = 0.014) and on F1 by +0.056 (Wilcoxon p = 0.085, not significant at $\alpha = 0.05$). Restricted to the 52 retrieval-hit questions, the pattern is the same in direction and magnitude (EM +0.096, p = 0.025; F1 +0.059, p = 0.099). Neither LSPM nor naive truncation differs significantly from the raw/unpruned answer on F1 in this smaller sample (all $p \geq 0.217$), consistent with, though not independently powered to confirm, the equivalence pattern established in Section 5.4. Because this check tests one ratio on one smaller, independent sample, we report these p-values as uncorrected and exploratory: they are a directionally consistent, complementary signal to the fully powered, Holm-Bonferroni-corrected r = 0.3 result in Section 5.4, not a second independent confirmation at the same statistical standard. We discuss what this does and does not add in Section 6.

6 Discussion

The tokenization-disparity measurement (5.1) establishes, with statistical confidence on the corpus tested, why Arabic RAG deployments face a KV-cache pressure problem that English deployments of the same architecture do not face to the same degree. The ratio-swept fidelity pilot (5.2) shows fidelity remains high across a range of compression ratios on the original small corpus. The analytical projection (5.5) translates the real, measured context reduction into a concrete systems-relevant unit (MiB of KV-cache per request) while remaining unambiguous that it is a projection, not a measurement.

The baseline comparison (5.3, re-tested at ARCD scale in 5.4) is the result that most shapes what this paper can and cannot claim, and we address it directly, including how our own reading of it has changed as we added statistical power. An earlier revision of this manuscript considered three, non-mutually-exclusive explanations for why LSPM's cross-encoder scoring showed no significant head-to-head advantage over naive truncation on an 8-question, low-redundancy corpus: a corpus effect (curated passages where "first sentences" and "top-scored sentences" overlap heavily), limited statistical power at n = 8, and the possibility that semantic scoring's advantage only manifests on longer, noisier, more redundant real-world retrieval results. A first ARCD re-test at n = 40 (five times the original pilot) left the head-to-head comparison still non-significant, which we reported honestly as unresolved rather than as a negative result, and flagged limited power as a live, not excluded, explanation: F1 favored LSPM by a consistent +0.078 to +0.090 across all three ratios at n = 40 without reaching significance, and a post-hoc power calculation on that effect size indicated roughly 115-140 questions per ratio would be needed for adequate power (previous revision, Section 8). We ran that fully powered re-test at n = 140 (Section 4.4/5.4), and it resolves the question at the most aggressive ratio: **at r = 0.3, LSPM significantly outperforms naive truncation head-to-head on F1 (+0.114, Holm-adjusted p = 0.0054), and is simultaneously**

shown statistically equivalent to the full unpruned answer by a predefined-margin equivalence test (TOST, p = 0.038), while naive truncation at the same ratio is significantly worse than the unpruned answer (adjusted p = 0.00044). This is the pattern the paper's central hypothesis predicts: when compression is aggressive enough that content selection actually matters, cross-encoder relevance scoring keeps the answer where blind length-matched truncation does not. At r = 0.5 and r = 0.7, where naive truncation retains more of the pool regardless of position, the same directional advantage persists (+0.062, +0.058 F1) but does not survive correction (adjusted p = 0.148 at both), so the comparison remains genuinely open at those two ratios rather than resolved either way. One important scope caveat carries over unchanged from the n = 40 re-test: the gold, answer-bearing paragraph is included in every pool by construction (Section 4.4), so recall@6 is 100% by design, and this experiment evaluates sentence-level pruning conditional on the answer-bearing passage already having been retrieved, not end-to-end retrieval robustness.

We had previously flagged, and withdrawn, an invalid inference from the n = 40 data: reading LSPM's non-significant difference from the unpruned answer alongside naive truncation's nominally significant one as evidence of an LSPM-specific "fidelity preservation" effect. That inference remains invalid for the reason we gave at the time (the asymmetry is algebraically the same paired quantity as the head-to-head test, not independent evidence), and we do not resurrect it here. The n = 140 result reported above is a different, more defensible claim: it comes directly from a pre-specified, corrected head-to-head test crossing significance after Holm-Bonferroni adjustment, not from an asymmetry between two separate vs-raw comparisons. As a secondary, exploratory, non-inferential observation offered to help explain the mechanism rather than as a finding in itself: recomputing which sentence actually contains each gold answer string at n = 140 shows naive truncation retains that sentence in 74.3% of questions at r = 0.3, rising to 80.7% at r = 0.5 and 87.9% at r = 0.7 (i.e., naive truncation loses the answer sentence most often precisely at the ratio where the head-to-head effect is significant), while the six-document pool's gold paragraph itself lands in the naive retriever's first position in 98/140 (70.0%) of cases after keyword-overlap re-ranking, consistent with the n = 40 re-test's 67.5%. This is a plausible mechanistic account of why the effect concentrates at r = 0.3 (naive truncation from the front of a re-ranked pool most often clips a genuinely different, non-first-ranked answer sentence exactly when the compression budget is tightest), and we present it as such, not as an independently tested claim.

On the systems side, StreamingLLM's attention sinks [5] and H2O's heavy-hitter eviction [6] both operate after tokens have already entered the KV cache; LSPM operates before the cache is populated at all, and at the most aggressive compression ratio tested, its content-aware selection now has statistically confirmed backing over the naive alternative for that role. Whether the same advantage holds at more moderate compression ratios remains an open, not yet statistically resolved, empirical question (Section 8).

The end-to-end retrieval check (4.9/5.7) asks a different, complementary question from the main re-test above: does the head-to-head advantage survive when retrieval itself can fail? With a real TF-IDF retriever

pulling from a 234-paragraph corpus rather than a curated, answer-guaranteed pool, recall@6 dropped to 86.7%, and every method's unconditional accuracy fell accordingly, exactly as expected when the correct text is genuinely absent for 13.3% of questions. Within that harder setting, LSPM's advantage over naive truncation at r = 0.3 held in direction and rough magnitude (EM +0.100 overall, +0.096 on the retrieval-hit subset), and was nominally significant on EM before any correction (p = 0.014 overall, p = 0.025 on the hit subset), though we are explicit that this is a single-ratio, n = 60, uncorrected result, not a second instance of the Section 5.4 Holm-Bonferroni-confirmed finding. We read it as a reassuring, non-contradictory signal that the mechanism established under answer-guaranteed retrieval does not simply evaporate once retrieval is allowed to fail, not as independent statistical proof of the same magnitude. The retrieval-miss subset (n = 8) is too small to support inference, but is worth flagging descriptively: LSPM's EM/F1 there (0.125/0.179) was not zero, unlike raw's and naive's (0.000/0.056 and 0.000/0.142), suggesting LSPM's sentence-level relevance scoring may extract partial signal from a wrong-but-topically-adjacent context even when the true answer text is absent, a specific, testable hypothesis for future work rather than a claim we make here.

7 Limitations and Threats to Validity

Scale of the preliminary experiments. The tokenization-disparity corpus (30 sentence pairs) and the original fidelity/baseline pilot (8 questions × 3 ratios × 2 methods = 48 real live-endpoint data points) remain small by the standards of a mature empirical NLP evaluation; they were sized to be fully hand-verifiable rather than scraped or machine-generated at unlimited scale. The ARCD ground-truth re-test (140 questions × 7 cells = 980 real live-endpoint data points, Section 4.4/5.4) was deliberately sized larger, from a post-hoc power calculation on an earlier 40-question pilot, specifically to resolve the head-to-head comparison; it succeeds in doing so at r = 0.3 (Holm-adjusted p = 0.0054) but remains underpowered, or the true effect remains genuinely smaller, at r = 0.5 and r = 0.7 (adjusted p = 0.148 at both), so this specific, narrower question is only partially closed.

No GPU-scale systems results in this submission. We have not measured throughput, TTFT, or KV-cache occupancy on a self-hosted, PagedAttention-based vLLM server under concurrent load, because no CUDA GPU was available at the time of this submission. Section 5.5's analytical projection is explicitly not a substitute for this measurement, only a motivating estimate built from real inputs.

The baseline comparison found a significant head-to-head LSPM advantage over naive truncation only at the most aggressive ratio tested (r = 0.3), after Holm-Bonferroni correction; at r = 0.5 and r = 0.7 the comparison remains statistically unresolved. On the original 8-question pilot (Section 5.3), all head-to-head p-values exceed 0.2. On the 140-question, ground-truth-scored ARCD re-test (Section 5.4), the r = 0.3 head-to-head comparison is significant after correction (adjusted p = 0.0054), and LSPM is additionally shown equivalent to the unpruned answer by TOST at r = 0.3 and r = 0.7. At r = 0.5 and r = 0.7 the head-to-head advantage remains directionally positive (+0.062, +0.058 F1) but does not

survive correction (adjusted p = 0.148 at both), so we cannot rule out that a true advantage exists at those ratios but remains below this test's detection threshold, nor can we claim one has been demonstrated there. Confirming or ruling out the head-to-head effect at r = 0.5/0.7 at still higher statistical power remains an open empirical question, though a materially smaller and lower-priority one than before this revision's re-test, and Section 8 discusses what further scale would be needed.

The r = 0.3 result rests on a single fully powered re-test, not an independent replication. We ran one properly powered ARCD re-test, not a second independent sample at the same scale; the usual caveats about a single significant result in a single dataset (this specific 140-question ARCD sample, this specific model, this specific retrieval pool construction) apply, and Section 8 identifies independent replication as a natural next step before treating the r = 0.3 finding as fully established.

The main ARCD re-test guarantees retrieval success by construction; the smaller end-to-end check (4.9/5.7) removes that guarantee but does not fully replace the main test's statistical power. The gold, answer-bearing paragraph is included in every six-document candidate pool in Section 4.4, so recall@6 is 100% by design there; that experiment measures how LSPM and naive truncation prune a pool known to contain the answer, not how they behave under realistic retrieval failure. Section 4.9/5.7 addresses this directly with a real TF-IDF retriever over a 234-paragraph corpus (recall@6 = 86.7%, so retrieval genuinely fails on 8/60 questions), and finds the same directional LSPM-vs-naive advantage there. But this check is a single ratio (r = 0.3), n = 60, uncorrected-p-value result, not a second instance of the Section 5.4 finding at the same statistical standard; a fully powered, Holm-Bonferroni-corrected end-to-end evaluation, ideally against a production-scale dense or hybrid retriever over a much larger real corpus, remains future work (Section 8).

Single generation model. All fidelity and baseline results use one instruction-tuned model (Llama-3.1-8B-Instruct) accessed through a hosted endpoint. Results may not transfer identically to other model families or sizes.

Retriever simplicity. The reference implementation's default retriever uses simple keyword overlap, and the ARCD re-test's noisier six-document pool (Section 4.4) is still a small, fixed candidate set assembled from a benchmark. The end-to-end check (Section 4.9) upgrades this to a real TF-IDF retriever over a 234-paragraph corpus, but that corpus and retriever are still far smaller and simpler than a production-scale dense or hybrid retrieval stack searching a large real corpus. LSPM is designed to be retriever-agnostic, but evaluation with a genuinely production-scale retrieval stack in front of it remains to be done.

Automatic metrics as fidelity proxies. The original pilot's BLEU, ROUGE-L, and BERTScore measure similarity to the unpruned answer, not correctness; the ARCD re-test (Section 4.4/5.4) addresses this specific gap by scoring against ARCD's real gold answers with Exact Match and token-F1. Both remain automatic proxies for human judgment, not human judgment itself, and Exact Match in particular is a strict, brittle metric at this sample size. A human evaluation component, or RAG-specific reference-free metrics such as RAGAS [37], would strengthen the claim in a follow-up study.

LSPM overhead measurement is CPU-only and single-request. Section 5.6's 57.8 ms figure does not characterize batched or concurrent-load latency, nor GPU-accelerated cross-encoder inference, which a production vLLM deployment would likely use.

8 Future Work

The fully powered head-to-head re-test called for in the previous revision has now been run (Section 4.4/5.4): a post-hoc power calculation on the n = 40 pilot's observed F1 differences (Cohen's $d \approx 0.24$-0.26 across ratios) indicated approximately 115-140 paired ARCD questions per ratio would be needed for 80% power, and we ran the identical ARCD pipeline at n = 140. This resolved the question at r = 0.3 (Holm-adjusted p = 0.0054, LSPM significantly better, and equivalent to unpruned by TOST) but left r = 0.5 and r = 0.7 still non-significant after correction (adjusted p = 0.148 at both). Two items are now the top remaining priorities. **First**, extending the power analysis to the two still-unresolved ratios specifically: the observed r = 0.5/0.7 effect sizes (+0.062, +0.058 F1) are smaller than the r = 0.3 effect that motivated the n = 140 target, so resolving them with 80% power would require a further sample-size increase beyond 140, which a follow-up study should compute directly from this revision's effect sizes rather than reusing the r = 0.3-derived target; independent replication of the r = 0.3 result on a fresh ARCD sample (or a second QA benchmark) would also strengthen that finding beyond a single fully powered run. **Second**, the GPU-scale throughput and KV-cache benchmark specified in Section 4.6: provisioning a CUDA GPU, running the full {0.2, 0.5, 0.8, 1.0} compression ratio × {1, 10, 25, 50, 100} concurrency matrix against a self-hosted vLLM server, and reporting throughput, latency percentiles, and measured (not projected) KV-cache occupancy, including for the naive-truncation condition, so the systems-level comparison is run for both methods, not just LSPM.

A first end-to-end retrieval check, with recall no longer guaranteed, has also now been run (Section 4.9/5.7): a real TF-IDF retriever over a 234-paragraph corpus, recall@6 = 86.7%, 60 questions at r = 0.3 only. It found the same directional LSPM-vs-naive advantage (EM +0.100, uncorrected p = 0.014), which is reassuring but explicitly not a fully powered or corrected confirmation. The natural next step is to scale this exact check up (a larger corpus, a production-scale dense or hybrid retriever in place of TF-IDF, and a sample size derived from this check's own observed effect size) to the same Holm-Bonferroni-corrected standard as Section 5.4, and to extend it across all three compression ratios rather than r = 0.3 alone.

Additional future work: incorporating RAGAS-style reference-free faithfulness and context-relevance metrics [37] alongside BLEU/ROUGE-L/BERTScore/EM/F1; a small human-evaluation component; quantifying LSPM's overhead under batched/concurrent load rather than the single-request CPU measurement in Section 5.6; quantifying the compositional benefit of combining LSPM with KV-cache-side management techniques such as StreamingLLM [5] or H2O [6]; testing the specific, currently-descriptive-only observation that LSPM retains partial signal when retrieval fails outright (Section 5.7's n = 8 retrieval-miss subset) with an adequately powered,

dedicated sample; and extending the dynamic compression-ratio controller (Section 3.4), currently an untested design proposal rather than a validated result, beyond linear interpolation to a learned policy evaluated under realistic, bursty production traffic. One implication of the r = 0.3-concentrated effect worth testing directly: since the dynamic controller is designed to tighten the compression ratio under memory pressure, a follow-up study could test whether it, in effect, automatically steers deployments toward the ratio regime (aggressive compression) where LSPM's advantage over naive truncation is now confirmed.

9 Conclusion

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, which we confirm with statistical significance on the corpus tested, across two independent tokenizers (aggregate ratio 1.69x-1.76x, bootstrap $p < 0.0001$, $n = 30$). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it with the same rigor and the same honesty regardless of outcome: a ratio-swept fidelity pilot (r = 0.3-0.7) shows preserved answer fidelity across the tested range on that pilot's small corpus, and a paired baseline comparison against naive length-matched truncation, run first on a small curated pilot ($n = 8$) and then re-tested on 140 real, ground-truth-scored ARCD questions (980 live generations) against a noisier, answer-guaranteed six-document pool, finds a statistically significant head-to-head advantage for LSPM's semantic scoring at the most aggressive compression ratio tested (r = 0.3: +0.114 F1, Holm-Bonferroni-adjusted $p = 0.0054$), where LSPM is additionally shown statistically equivalent to the unpruned answer (TOST, $p = 0.038$) while naive truncation is significantly worse than the unpruned answer (adjusted $p = 0.0004$). At the more moderate ratios (r = 0.5, 0.7) the same directional advantage does not survive correction, so the comparison remains genuinely open there. A smaller, complementary check that replaces the answer-guaranteed retrieval pool with a real TF-IDF retriever over a larger corpus (recall@6 = 86.7%, so retrieval genuinely fails on some questions) finds the same directional advantage at r = 0.3, reported honestly as a single-ratio, uncorrected, exploratory result rather than a second fully powered confirmation. We report this exact pattern, resolved with statistical confidence at one ratio, open at two others, and directionally reinforced under harder, more realistic retrieval, rather than rounding it up to a uniform win or down to a uniform null, having caught and corrected our own invalid overreading of an earlier, smaller-sample asymmetry during a prior revision. We provide an analytical, explicitly-labeled projection connecting the real measured context reduction to KV-cache bytes saved (9.7-19.1 MiB per request depending on ratio), and we specify, in full, the GPU-scale benchmark and the ratio-specific higher-power re-test still needed to convert this paper's architecture and preliminary evidence into a fully validated systems result. We release the complete implementation, all raw data from every round of experiments including both ARCD re-tests and the end-to-end retrieval check, and a live demonstration interface to support independent reproduction and extension of every claim in this paper, including the significant, null, and inconclusive results alike.

Statements and Declarations

Data availability. The complete source code (LSPM middleware, evaluation scripts, statistical analysis scripts, benchmarking harness, and demonstration interface) is publicly available at https://github.com/Akramtaha98/vllm-arabic-rag (commit 22e36ac83bb0b074779db5b3607bbcb57ab90368). The 30-pair Arabic-English tokenization corpus, the 8-question/10-document fidelity and baseline pilot dataset (all 48 ratio×method cells across its single- and multi-ratio phases), the ARCD ground-truth re-test dataset and code (`build_arcd_eval_set.py`, `arabic_em_f1.py`, `run_arcd_pilot.py`, `analyze_arcd_results.py`, and the full 980-row `arcd_results.jsonl` output at n = 140, which contains the original n = 40 sample as its first 280 rows), the end-to-end retrieval check dataset and code (`build_e2e_retrieval_eval.py`, `run_e2e_retrieval_pilot.py`, `analyze_e2e_retrieval_results.py`, and the 180-row `e2e_retrieval_results.jsonl` output), and all raw experimental outputs (tokenization CSVs, raw/pruned/naive answer triples, fidelity metric scores, bootstrap analysis outputs, the Holm-Bonferroni and TOST statistical outputs, and the analytical KV-cache estimate script and its output) will be added to the same repository, with a tagged release and a Zenodo-archived DOI snapshot, before final submission.

Ethics declaration. This study did not involve human participants, personal data, or clinical data. The parallel-corpus and pilot-corpus text was authored by the researcher for evaluation purposes and describes only publicly available, non-sensitive institutional information. No ethics committee approval was required.

Author contributions (CRediT). Akram Taha: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Conflict of interest. The author declares no conflict of interest.

Funding. This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

AI usage disclosure. See Section 4.8 for the full disclosure of AI assistance used in this project.

References

[1] A. Vaswani et al., "Attention Is All You Need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems

Principles (SOSP), 2023, doi: 10.1145/3600006.3613165

[5] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[6] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023, doi: 10.52202/075280-1506

[7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in Proc. 16th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022.

[8] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 1715–1725, doi: 10.18653/v1/P16-1162

[9] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, doi: 10.18653/v1/2023.emnlp-main.825

[10] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, doi: 10.18653/v1/2023.emnlp-main.391

[11] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[12] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2020, doi: 10.1145/3397271.3401075

[13] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019, doi: 10.48550/arXiv.1901.04085

[14] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023, doi: 10.48550/arXiv.2312.10997

[15] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. Int. Conf.

Learning Representations (ICLR), 2024.

[16] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019

[17] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, doi: 10.18653/v1/2020.emnlp-main.550

[18] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638

[19] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, doi: 10.18653/v1/D19-1410

[20] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," arXiv preprint arXiv:2402.05672, 2024, doi: 10.48550/arXiv.2402.05672

[21] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," Transactions of the Association for Computational Linguistics, vol. 11, pp. 1114–1131, 2023, doi: 10.1162/tacl_a_00595

[22] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL), 2023, doi: 10.18653/v1/2023.eacl-main.148

[23] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," in Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020.

[24] W. Antoun, F. Baly, and H. Hajj, "AraGPT2: Pre-Trained Transformer for Arabic Language Generation," in Proc. 6th Arabic Natural Language Processing Workshop (WANLP), 2021.

[25] H. Mozannar, K. El Hajal, E. Maamary, and H. Hajj, "Neural Arabic Question Answering," in Proc. 4th Arabic Natural Language Processing Workshop (WANLP), 2019, doi: 10.18653/v1/W19-4612

[26] N. Sengupta et al., "Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative Large Language Models," arXiv preprint arXiv:2308.16149, 2023, doi: 10.48550/arXiv.2308.16149

[27] A. Huang et al., "ACVA: Arabic Cultural and Value Alignment Benchmark," Hugging Face Dataset, 2024. [Online]. Available: https://huggingface.co/datasets/FreedomIntelligence/ACVA-Arabic-Cultural-Value-Alignment

[28] F. Koto et al., "ArabicMMLU: Assessing Massive Multitask Language Understanding in Arabic," 2024, doi: 10.18653/v1/2024.findings-acl.334

[29] O. Obeid et al., "CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," in Proc. 12th Language Resources and Evaluation Conf. (LREC), 2020, pp. 7022–7032.

[30] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022, doi: 10.52202/068431-1189

[31] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[32] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," in Proc. Machine Learning and Systems (MLSys), 2024.

[33] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023.

[34] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318, doi: 10.3115/1073083.1073135

[35] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Text Summarization Branches Out: Proc. ACL Workshop, 2004, pp. 74–81.

[36] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. Int. Conf. Learning Representations (ICLR), 2020.

[37] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), 2024, doi: 10.18653/v1/2024.eacl-demo.16

[38] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall/CRC, 1993, doi: 10.1201/9780429246593

[39] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021, doi: 10.1109/TBDATA.2019.2921572

[40] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv preprint arXiv:2402.03216, 2024, doi: 10.48550/arXiv.2402.03216

[41] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024, doi: 10.48550/arXiv.2412.15115

[42] AI@Meta, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024, doi: 10.48550/arXiv.2407.21783